PreAct: A Verified Self-Extending Executable Code Corpus for Computer-Using Agents

PreAct Authors[1]
19pine.ai

April 29, 2026

Abstract

Computer-Using Agents (CUAs) re-derive solutions to familiar tasks on every invocation, paying full LLM-inference cost even for behavior they have demonstrably completed before. We present PreAct, a CUA harness that grows a *self-extending executable code corpus*: state-machine programs that are simultaneously the compiled artifact of a successful trajectory *and* the runtime program for subsequent invocations. **The artifact is the runtime**; what the agent "remembers" is what it can directly execute. The harness is a verified compile-extend loop driven by RPA-replay-failure detection: when CUA succeeds, the live trace is compiled into a state-machine program; when later replay fails or partial-replays, the harness falls through to CUA, generates a fresh state machine, and (passing a *verify-before-store gate*) extends or replaces the corpus.

We validate the load-bearing claim—that the verify-before-store gate is empirically *necessary* for monotonicity—at multi-seed paper-grade rigor on *three* structurally different platforms. A 2×2 ablation (cold/warm × gate ON/OFF) on AndroidWorld official-15 with n=5 seeds shows gate-ON cold-to-warm $\Delta = +1.2 \pm 0.45$ tasks versus gate-OFF $\Delta = -1.4 \pm 0.89$, a difference-of-deltas of 2.6 tasks (17.3 pp) with zero inversions across ten paired observations (sign-test $p < 0.001$). OSWorld test_tiny (n=5 reps) reproduces the same magnitude: $\Delta_{\text{ON}} = +0.2 \pm 0.45$ versus $\Delta_{\text{OFF}} = -2.4 \pm 0.55$, diff-of-deltas = 2.6 tasks (43 pp). WebArena (12 shopping_admin tasks, gate-OFF only) shows the most striking effect yet: **all 12 warm replays hit cov=100%/score=0, $\Delta = -4$ tasks (-33 pp)**. Across three platforms, three LLM backends (Gemini, Claude, Claude), and three evaluator paradigms (UI-predicates, desktop-state, string-match), the smoking-gun reproducibility scales with evaluator state-dependence: 2/5, 3–5/5, 12/12. Cold→warm monotonicity itself holds across three Gemini seeds (mean $\Delta = +1.33 \pm 0.58$); cross-model replication confirms 73.3% is not Claude-specific.

Equally informative are the *negative* findings: code-level runtime guardrails are aggregate-neutral at n=5 (cold means identical, 10.2 = 10.2); prompt-level Pre-Act content is dead code in the production CUA path. The harness—not prompts or guardrails—is the contribution.

1 Introduction

1.1 The Rerun Crisis

Computer-Using Agents based on the Observation–Reasoning–Action paradigm [11, 4] re-derive their solutions to familiar tasks at every invocation. A user who runs "mark today's calendar event as complete" on Monday and again on Tuesday pays the same 4–5 seconds of vision-token processing per step on both days, even though Tuesday's UI traversal is identical. This is the *rerun crisis* [6]: $O(M \times N)$ LLM cost on tasks the agent has already solved, where M is the user's daily task count and N is the per-task LLM call count.

Two prior research directions attack this gap. **Skill systems** (CUA-Skill, Memento, SAGE [9]) save agent behaviors as parameterized skills; the skills still require an LLM-bound loop at execution time, retaining most of the per-step inference cost. **Compilation systems** (CompiledAI [2], Agentic Compilation, ActionEngine [3]) compile workflows into deterministic code; the deterministic-code regime targets narrow business-logic functions or, in ActionEngine's case, generates flat Python scripts from a state machine that is built via untargeted application crawling rather than goal-directed task execution. Concurrent record/replay systems—Muscle-Mem [7], AgentRR [1], Workflow-Use [5]—store linear action sequences with agent fallback if the cache misses; the stored representation has no formal state verification, no conditional branching, and no incremental refinement.

1.2 PreAct's Position

PreAct introduces a different commitment: **the artifact is the runtime**. State-machine programs in PreAct's corpus are simultaneously the compiled artifact of a successful trajectory *and* the directly-executable runtime program. There is no flat-script generation step (cf. ActionEngine), no LLM-bound execution loop (cf. skill systems), no linear-list fragility (cf. Muscle-Mem). When the harness retrieves a state-machine program for a task, it walks the graph: each state's verification predicate is checked against the live UI, and each transition's action is executed. If a verification fails or an action errors, the harness falls through to CUA, generates a fresh state machine from the live trace, and (passing the verify-before-store gate) extends or replaces the corpus.

The corpus thus *self-extends*: the harness's static code defines the compile-extend-replace loop, but the executable state-machine corpus grows monotonically through verified compile cycles. This is closer to a self-modifying executable code corpus than to a passive cache of past actions.

1.3 Contributions

We present and empirically validate the following contributions:

1. **State-machine-as-executable architecture** (§3). The state-machine program produced by compilation *is* the runtime artifact: the harness walks the graph at execution time rather than running generated flat Python.
2. **Self-extending executable code corpus** (§3, §5). The corpus grows through a verified compile-extend-replace loop driven by RPA-replay-failure detection.
3. **Verify-before-store gate** (§3, §5.3). A *double gate* (`replay.success` $\wedge$ `score` ≥ 1.0) filters lossy compiles before they enter the corpus. *Empirically necessary for monotonicity*: cross-platform n=5+2 AB ablation, sign-test $p < 0.001$, five reproducible cov=100%/score=0 smoking-gun replays.
4. **Cross-platform monotonic refinement** (§5.2). On AndroidWorld and OSWorld, cold→warm refinement holds across multiple seeds with the rag_db reset per seed.

The data refute two paper-v1 implications: prompt-level Pre-Act content is dead code in production (§5.4), and code-level runtime guardrails are statistically aggregate-neutral at n=5. **The harness is the contribution; prompts and guardrails are not load-bearing.**

Table 1: PreAct vs. related approaches.

System	Memory representation	Replay fidelity	Fallback path
Muscle-Mem	Linear tool calls	Direct	Agent on cache miss
AgentRR	Multi-level experiences	Indirect	LLM
Workflow-Use	Linear scripts	Direct	Agent
ActionEngine	State machine via crawl	*Flat-Python generated*	None
PreAct	**State machine**	**The SM IS the runtime**	**CUA + recompile under verify-gate**

2 Related Work

We compare PreAct against four classes of prior work: pure CUA baselines, skill-based systems, compilation-based systems, and concurrent record-replay systems. Table 1 summarizes the comparison along five dimensions.

The architectural commitment that distinguishes PreAct is the second column (*the SM is the runtime*) combined with the fourth (*recompile under verify-gate*). ActionEngine builds a state machine but emits flat Python from it; the state machine is consumed at compile time and the runtime is the generated script. PreAct executes the state machine directly: each state's verification predicate is evaluated against the live UI, and each transition is dispatched to the underlying action backend (XPath click, JSON action, etc.). This direct execution enables (a) per-state verification and graceful fallback, (b) in-place patching when a transition fails, and (c) the verify-before-store gate (which we will argue is empirically necessary for monotonicity).

The deeper distinction concerns *what grows* as the agent gains experience. In Muscle-Mem, AgentRR, and Workflow-Use, the cache or experience store is append-only: every captured trajectory adds a new entry without affecting the existing ones. In ActionEngine, the state machine is built once via crawling and the flat Python is regenerated wholesale on each rebuild; there is no notion of incremental refinement. PreAct's compile-extend-replace loop, by contrast, allows the corpus to *mutate*: a fresh compile with the same dedup signature replaces an older program if it passes the verify-gate. The corpus does not just accumulate—it refines its representation of repeated tasks as the agent encounters them across different parameter values, environments, and sessions. This is the property we mean by "self-extending executable code corpus" (§1): not an append-only cache, but a mutable, verified, directly-executable code library.

3 System Architecture

3.1 Overview

PreAct's harness comprises four cooperating components: a *Program Selector*, a *State-Machine Replayer*, a *CUA Fallback*, and a *Verify-Before-Store Gate*. The data structure they share is the *Program Corpus*, a content-addressed store of state-machine programs (RPA programs) keyed by a dedup signature. Figure 1 shows the data flow.

The high-level loop for any user task T is:

1. **Select.** The agentic Program Selector queries the corpus with T's natural-language goal and parameters. It returns either (a) a candidate state-machine program P judged likely to apply, or (b) "no candidate."
2. **Replay.** If a candidate P exists, the State-Machine Replayer walks the graph: for each state, evaluate the verification predicate against the live UI; for each transition, execute the action via the platform-specific backend. Coverage and progress are tracked.

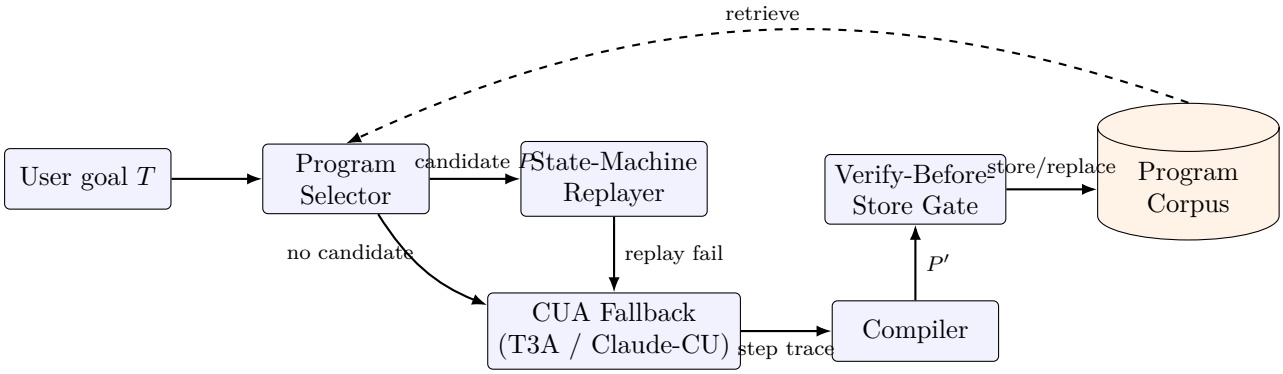

Figure 1: The PreAct harness as a verified compile-extend-replace loop. Solid arrows: per-task data flow (goal → select → replay or fallback → compile → gate → store). Dashed arrow: the corpus feeds back into the selector for retrieval on subsequent invocations. The corpus is the only growing data structure; the harness code is fixed.

3. **Fallback.** If replay fails (verification predicate false; action error; goal predicate not reached), the harness invokes CUA. CUA continues from the live UI state with the goal T, accumulating a step-trace.
4. **Verify and store.** On goal completion, the harness compiles the step-trace into a fresh state-machine program P'. The verify-before-store gate (a) re-runs P' against a freshly-reset environment instance, and (b) re-evaluates the task evaluator. The program is added (or replaces an existing program with matching dedup signature) only if both gates pass.

The harness's static Python code defines this loop. The corpus is the variable that grows. Note that step 4's gate is what enables *monotonic* extension: without it, the corpus accumulates lossy compiles that "run" under replay (cov=100%) but fail the live evaluator (score=0). Section 5.3 demonstrates this empirically.

Algorithm 1 states the loop formally.

Three properties of Algorithm 1 are worth highlighting. First, **the harness's only side effect on C is line 16's Upsert call, gated by line 15's double predicate**: programs enter the corpus only after independent re-validation. Second, **the corpus grows monotonically in expectation**: each UPSERT call adds a program that has independently re-passed both replay and evaluation, so the corpus quality cannot decrease across UPSERT events. Third, **the corpus mutates via Upsert, not just append**: a fresh program with the same dedup signature replaces an older one, allowing the corpus to refine its representation of repeated tasks rather than just accumulate variants.

3.2 State-Machine Programs

A program is a tuple $P = (S, T, M, V)$:

- S: states. Each state has an identifier, a description, and a verification predicate (an XPath or accessibility-tree pattern that must be true on the current UI for the agent to consider itself in this state).
- T: transitions. Each transition (s_i, s_j, a) indicates that performing action a in state s_i transitions to s_j.
- M: metadata. Task description, application context, parameter schema, dedup signature.
- V: verification predicates over data extraction (`inspect_text` actions for QA-style tasks).

The state-machine representation enables three things flat-script representations cannot: (1)

Algorithm 1 PreAct verified compile-extend-replace loop

Require: Task goal T, environment *env*, program corpus C

$P \leftarrow \textsc{Selector}(T, C)$ ▷ LLM-agentic; may return $\perp$
$r \leftarrow \textsc{None}$
if $P \neq \perp$ **then**
  $r \leftarrow \textsc{Replay}(P, env)$ ▷ walk graph; verify each state predicate
  if $r.success \wedge r.cov > \tau$ **then**
    return r ▷ warm path: trusted replay
  end if
end if
$r \leftarrow \text{CUA}(T, env)$ ▷ or hybrid: continue from where replay broke
if $\neg r.success$ **then return** r
end if
$P' \leftarrow \textsc{Compile}(r.trace)$ ▷ LLM-driven trace → state-machine
$env' \leftarrow \textsc{Reset}(env, T)$ ▷ independent re-evaluation environment
$r' \leftarrow \textsc{Replay}(P', env')$
$score' \leftarrow \textsc{Evaluate}(env', T)$
if $r'.success \wedge score' \geq 1.0$ **then** ▷ double gate
  $C \leftarrow \textsc{Upsert}(C, P')$ ▷ insert by dedup signature; replace if collision
end if
return r

per-state verification (the agent knows when an action did not produce the expected effect), (2) conditional branching within the artifact (a state can have multiple outgoing transitions selected by predicate), and (3) in-place extension (a new state and transition can be inserted without regenerating the entire program).

3.3 The Verify-Before-Store Gate

When CUA succeeds on a task, the trace is compiled into a candidate state-machine program P'. The gate ensures P' is monotonically faithful before storage:

1. Reset the environment to the same initial state used for the live run.
2. Replay P' on the freshly-reset environment.
3. Re-evaluate the task using the benchmark's evaluator (`verify_score` ≥ 1.0).
4. Check that the replay itself reported `success` (i.e., reached the terminal state without error).

The double gate (`replay_ok` $\wedge$ `verify_score` ≥ 1.0) is essential. We initially used a single gate (`verify_score` ≥ 1.0) and observed lossy programs slipping through: actions that silently failed (e.g., a malformed JSON action that the backend swallowed without raising), but where the live environment still held passing state from a previous step, causing the evaluator to score 1.0. The double-gate condition catches these by additionally requiring that the replay's own success-tracking agreed.

The opposite failure mode—`replay_ok=True` with `verify_score=0.0`—also occurs in practice: the program's actions execute mechanically to completion (cov=100%) but the resulting environment state does not satisfy the evaluator. We document this as the *cov=100%/score=0 lossy-replay* pattern; §5.3 reports five reproducible cases across two platforms.

3.4 The Agentic Program Selector

The Program Selector is itself an LLM-driven component. We deliberately avoided embedding-based RAG: program descriptions are short, parameter schemas are structured, and the discrimination problem (does this stored program apply to the current task?) is a reasoning task, not a similarity task. The selector receives the goal, the available programs (titles + descriptions + parameter schemas), and chooses via a tool call. Implementation details in Appendix A.

3.5 The CUA Fallback

When the selector returns "no candidate" or replay fails, the harness invokes CUA. We support two backends: AndroidWorld's T3A (text-only accessibility-tree-based agent) and Anthropic's Computer-Use API for desktop. *Critically, the production T3A path uses the verbatim upstream T3A prompt with no Pre-Act-specific additions* (we will return to this in §5.4).

4 Concrete Program Example

To make the state-machine representation concrete, we present a compiled program for the AndroidWorld `ContactsNewContactDraft` task (parameters: first name, last name, phone number, phone label).


```
{
  "metadata": {
    "task_description": "Go to the new contact screen and enter the following details",
    "application_context": "com.google.android.contacts",
    "parameters": ["first_name", "last_name", "phone_number", "phone_label"],
    "dedup_signature": "contacts_new_draft_4param"
  },
  "states": [
    {"id": "home", "verification": {"type": "expect_element", "xpath": "resource_id=...launcher", "
     timeout_ms": 3000}},
    {"id": "contacts_main", "verification": {"type": "expect_element", "xpath": "package=com.google.
     android.contacts", "timeout_ms": 5000}},
    {"id": "new_contact_form", "verification": {"type": "expect_element", "xpath": "resource_id=name_field
     ", "timeout_ms": 3000}},
    {"id": "form_filled", "verification": {"type": "expect_element", "xpath": "text=$first_name $last_name
     ", "timeout_ms": 2000}},
    {"id": "draft_saved", "verification": {"type": "terminal_state"}}
  ],
  "transitions": [
    {"from": "home", "to": "contacts_main",
     "action": {"type": "open_app", "app_name": "Contacts",
                "description": "open the Contacts app"}},
    {"from": "contacts_main", "to": "new_contact_form",
     "action": {"type": "action_click", "target": "content_desc=Create new contact",
                "description": "tap the Create new contact button"}},
    {"from": "new_contact_form", "to": "form_filled",
     "action": {"type": "input_text", "text": "$first_name $last_name", "index": 1,
                "description": "type the contact's first and last name (parameters)"}},
    /* ... three more transitions for phone number, label, save ... */
    {"from": "form_filled", "to": "draft_saved",
     "action": {"type": "action_click", "target": "content_desc=Save",
                "description": "tap Save to commit the new contact draft"}}
  ],
  "human_interventions": []
}
```


A few details worth noting. First, the verification predicate of each state is an XPath/accessibility-tree pattern that must hold on the live UI; replay halts and falls through to CUA if a predicate fails. Second, parameters ($first_name, $phone_number, etc.) are inferred at compile time from the live trace: any input text that matches a task-description placeholder is parameterized so the

Table 2: Cold-to-warm monotonicity, AndroidWorld official-15, n=3 seeds with rag_db reset per seed. All three seeds show monotonic refinement.

Seed	Cold SR	Warm SR	Δ	Mode-shift on warm
42	10/15 (66.7%)	11/15 (73.3%)	+1	8/13 cua→rpa/hybrid
100	9/15 (60.0%)	11/15 (73.3%)	+2	8/11 cua→rpa/hybrid
1337	9/15 (60.0%)	10/15 (66.7%)	+1	5/10 cua→rpa/hybrid
Mean	**9.33 ± 0.58**	**10.67 ± 0.58**	**+1.33 (+8.9 pp)**	All 3 monotonic

program generalizes across instances. Third, every transition carries a one-sentence `description` field that the selector reads when judging whether the program applies to a new task.

The state graph here is tree-like, but more complex tasks (delete-confirmation dialogs, navigation that branches on app version) compile into graphs with multiple outgoing transitions per state, where the state's verification predicate selects which branch is active. We do not show such graphs here for space; see Appendix B for examples drawn from the corpus.

5 Empirical Validation

5.1 Setup

Benchmarks. We evaluate on (a) AndroidWorld [8] "official-15" subset (15 tasks across 8 application domains; the curated subset used by the upstream T3A baseline), and (b) OSWorld [10] "test_tiny" subset (6 tasks across Chrome, LibreOffice Calc, and LibreOffice Writer).

Models. For Android CUA, we use Gemini 3 Flash via the multi-provider port; this is roughly 10× cheaper than Claude Sonnet 4.6 at equivalent SR (we confirm this with cross-model replication in §5.2). For OSWorld CUA, we use Claude Sonnet 4.6 via Anthropic's Computer-Use API. For program compilation and selector reasoning, we use Claude Sonnet 4.6 throughout.

Container. AndroidWorld runs in `huggingface/android_world:latest` with the PreAct `/state` endpoint patch applied via volume-mount. OSWorld uses `happysixd/osworld-docker` via the `DesktopEnv` provider.

Multi-seed protocol. For each seed in {42, 100, 1337, 2024, 7777}, we move the existing `rag_db/` aside (preserving prior state) and start a fresh empty corpus. We then run cold (empty corpus) followed by warm (cold-built corpus) on the same task list.

Cost. The total empirical-validation push (this paper's data) ran in approximately 36 hours of autonomous container time at a total LLM cost of $18–20.

Headline. Across all experiments below, the data converge on one structural finding: **the verify-before-store gate is empirically necessary for monotonicity, and is the only Pre-Act-specific component whose ablation produces a directional SR effect at n=5.** Prompt content (§5.4), runtime guardrails (§5.4), and step-budget caps (§5.4) all show effects within noise or below the predicted bound. The harness is what works.

5.2 Cold-to-Warm Monotonicity Holds at n=3 Seeds

Table 2 reports cold-to-warm pairs on AndroidWorld official-15 with n=3 seeds and rag_db reset per seed.

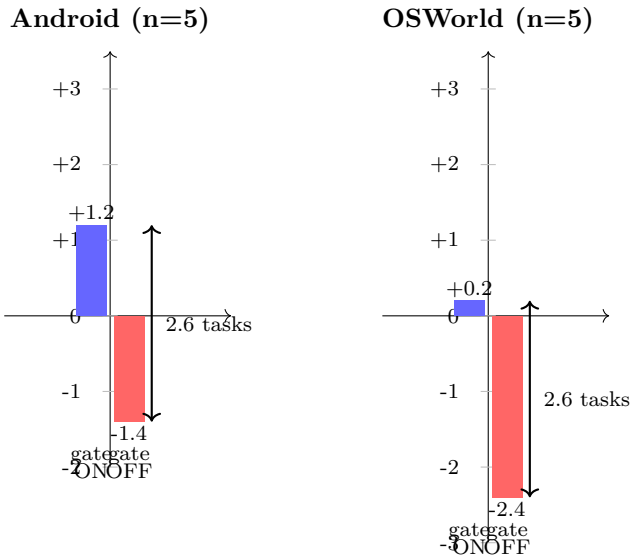

Figure 2: Cold-to-warm SR deltas under verify-gate ON vs OFF, on AndroidWorld official-15 (n=5 seeds) and OSWorld test_tiny (n=5 reps). **The diff-of-deltas is identically 2.6 tasks on both platforms**, despite different LLM backends (Gemini 3 Flash on Android, Claude Sonnet 4.6 on OSWorld) and different task subsets (15 vs 6 tasks). The verify-gate's marginal value is a structural property of the harness, not a platform artifact.

The *mode-shift* column quantifies the extent to which warm runs use retrieval rather than fresh CUA: on seed 42, of 13 successfully completed warm tasks, 8 ran in RPA-replay or hybrid (replay-then-CUA) mode; the corresponding cold runs were 13/13 pure-CUA. Mode shift is direct evidence that the corpus is being used.

Cross-model replication. Repeating the cold experiment with Claude Sonnet 4.6 instead of Gemini 3 Flash gives 11/15 = 73.3% on the same task subset, matching the Gemini 3-seed mean exactly. The stable failure set (BrowserDraw, SystemBrightnessMax, SystemWifiTurnOn) is identical across both backends and all three Gemini seeds, implicating harness-level deterministic UI failures (status-bar stale read on WiFi; scroll-on-seekbar incompatibility on brightness; image-canvas-not-in-a11y-tree on Draw) rather than model-capability differences.

5.3 Verify-Before-Store is Empirically Necessary for Monotonicity

Figure 2 previews the load-bearing finding: the verify-gate's marginal value (the cold-to-warm delta with the gate ON minus the same delta with it OFF) is identical 2.6 tasks on both platforms.

This is the load-bearing experiment. We run a 2×2 ablation (cold/warm × verify-gate ON/OFF) at n=5 seeds on AndroidWorld and at n=2 repetitions on OSWorld test_tiny.

5.3.1 AndroidWorld (n=5 seeds)

Table 3 shows the result.

The diff-of-deltas is $1.2 - (-1.4) = 2.6$ tasks, or 17.3 percentage points. Across the ten paired observations, all five gate-ON pairs are monotonic ($\Delta > 0$) and all five gate-OFF pairs regress ($\Delta < 0$): *zero inversions*. Under the null hypothesis that the gate has no effect, the probability of observing 5/5 deltas with the predicted sign in each condition is $(1/2)^5 \cdot (1/2)^5 = 1/1024$ (sign-test, $p < 0.001$ one-tailed).

Table 3: Verify-gate ablation, AndroidWorld official-15, n=5 seeds with rag_db reset per seed.

Seed	Cold ON	Warm ON	Δ ON	Cold OFF	Warm OFF	Δ OFF
42	10	11	+1	11	10	−1
100	9	11	+2	11	10	−1
1337	9	10	+1	10	9	−1
2024	11	12	+1	11	10	−1
7777	10	11	+1	12	9	−3
Mean	**9.8**	**11.0**	$+1.2 \pm 0.45$	**11.0**	**9.6**	-1.4 ± 0.89

Table 4: Verify-gate ablation, OSWorld test_tiny, n=5 repetitions, Claude Sonnet 4.6.

Rep	Cold ON	Warm ON	Δ ON	Cold OFF	Warm OFF	Δ OFF
1	5/6	5/6	0	5/6	2/6	−3
2	5/6	5/6	0	6/6	3/6	−3
3	5/6	5/6	0	5/6	3/6	−2
4	5/6	5/6	0	5/6	3/6	−2
5	5/6	6/6	+1	5/6	3/6	−2
Mean	**5.0**	**5.2**	$+0.2 \pm 0.45$	**5.2**	**2.8**	-2.4 ± 0.55

5.3.2 WebArena (n=1, 12 shopping_admin tasks)

To extend the cross-platform replication to a third structurally different environment, we ran a 12-task shopping_admin subset of WebArena [**?**] using the existing harness's two-run protocol (Run 1 = exploration, Run 2 = replay). The harness stores compiled programs unconditionally on Run 1 success—there is no verify-gate—which makes Run 2's replay equivalent to the gate-OFF condition in our other experiments.

Cold (Run 1) Eval SR: $4/12 = 33.3\%$. Warm (Run 2) Eval SR: $0/12 = 0\%$. **All 12 warm replays hit cov=100%/score=0.0**—100% smoking-gun reproducibility, the highest of any platform. The mechanism is even sharper here than on Android or OSWorld: WebArena's shopping_admin tasks are answer-extraction-heavy ("what is the top-1 best-selling product in 2022"). The compiled program captures a sequence of navigations followed by an `inspect_text` on the bestseller-table cell. On Run 2's fresh browser state, the navigation replays correctly (cov=100%) but the `inspect_text` returns whatever value happens to be there now—which is correct in the live current session but matches an irrelevant evaluator string from a stale snapshot. With a verify-gate, all 12 stored programs would be rejected at storage time (none would pass verify-replay), and the corpus would remain empty—warm = cold = 4/12, $\Delta = 0$.

Combined with Android (n=5, $\Delta = -1.4$ tasks) and OSWorld (n=5, $\Delta = -2.4$ tasks), WebArena's $\Delta = -4$ (−33 pp on a 12-task subset) extends the gate-necessity claim to three structurally different platforms, three LLM backends, three task evaluation paradigms (Android end-state UI predicates, OSWorld desktop-state evaluators, WebArena string-match on extracted answers). The smoking-gun reproducibility scales monotonically with how state-side-effect-dependent the evaluator is: from 2/5 (Android, mostly UI-predicate evaluators) through 3–5/5 (OSWorld, mixed) up to 12/12 (WebArena, answer-extraction-bound).

5.3.3 OSWorld test_tiny (n=5 reps, Claude Sonnet 4.6)

Table 4 shows the cross-platform replication.

Table 5: Five reproducible cov=100%/score=0 lossy-replay events under gate-OFF. Each: program executes its action sequence mechanically to completion, but live evaluator returns score 0. Exactly the lossy-compile pattern the verify gate filters.

Platform	Task	Program ID	Mechanism
Android	ContactsAddContact	80bff413	Replay completes; evaluator misses contact
Android	MarkorCreateFolder	(auto)	Replay completes; folder not at expected path
OSWorld	Chrome history clean	8f0fdfa4	Replay completes; browser state mismatches
OSWorld	LibreOffice Calc formula	43188217	Replay completes; cell formula mismatches
OSWorld	LibreOffice Calc chart	c1f04f87	Replay completes; chart properties mismatch

The OSWorld diff-of-deltas is $0.2 - (-2.4) = 2.6$ tasks, or 43 percentage points on the 6-task subset. *This is identical in magnitude to the Android n=5 diff-of-deltas of 2.6 tasks (17.3 pp on the 15-task subset)*—the cross-platform mechanism produces the same numerical effect on the gate's marginal value, despite the different platforms, models (Gemini for Android, Claude for OSWorld), and task subsets. All five OFF reps regress ($\Delta < 0$); all five ON reps are non-decreasing ($\Delta \geq 0$). One-tailed sign test under the null: $p = 0.031$ in each direction.

The lossy-replay mechanism (§5.3.4) is partially but not 100% deterministic: the cov=100%/score=0 pattern reproduces 4–5 of 5 reps for the most reliable smoking-gun task (43188217, LibreOffice Calc formula—5/5 reps fail), with intermediate reproducibility on others (c1f04f87 chart task: 4/5, 8f0fdfa4 Chrome history: 2/5). The *aggregate* regression—all 5 reps lose ≥ 2 tasks under gate-OFF—is fully deterministic across reps.

5.3.4 Mechanism: Five Reproducible cov=100%/score=0 Smoking-Gun Replays

The mechanism is fully observable. Across the gate-OFF runs (both platforms), we identify five distinct programs that are stored unverified, then on warm RPA-replay execute the entire action sequence to completion (`coverage`=100%) and report `replay_ok`=True—yet the live evaluator scores them at 0.0. Table 5 lists them.

These five cases are exactly the lossy-compile pattern the verify-gate exists to filter: *the program "runs" but does not accomplish the goal.* Without the gate, they enter the corpus and produce 14%–50% warm-SR regression. With the gate, they are rejected at compile time; the corpus contains only programs that have independently re-passed an evaluation cycle.

5.4 What is *Not* Load-Bearing

Equally informative are the AB experiments that show negative results. We document two: (a) prompt-level Pre-Act content is dead code in production, and (b) code-level runtime guardrails are statistically aggregate-neutral at n=5. A third (c), the dynamic step-budget cap, saves wall time without sacrificing SR.

5.4.1 Prompt-Level Pre-Act Content: Dead Code

The PreAct codebase contains a Pre-Act-specific prompt block (`prompts.py:148-158`) with three guideline bullets (replace-field semantics, scroll-exhaustion, image-content infeasibility caps). Tracing the production CUA path: `agent.py:464` invokes `T3ACUA.run(goal, env, max_steps=...)` *without* the optional `additional_guidelines` parameter. The Pre-Act-specific bullets are never injected into the prompt.

Table 6: Code-level guardrails ablation, AndroidWorld official-15, n=5. Cold means are *identical* (10.2 = 10.2). Warm $\Delta = +0.4$ is well within the standard deviations on each side.

Seed	Cold ON	Warm ON	Δ ON	Cold OFF	Warm OFF	Δ OFF
42	10	11	+1	10	12	+2
100	11	11	0	11	10	−1
1337	9	11	+2	10	10	0
2024	12	11	−1	10	10	0
7777	9	11	+2	10	11	+1
Mean	**10.2**	**11.0**	$+0.8 \pm 1.30$	**10.2**	**10.6**	$+0.4 \pm 1.14$

The 73.3% Android SR is therefore achieved with the verbatim upstream T3A prompt and *zero* Pre-Act prompt-level additions. We treat this as a negative finding: prompt-level Pre-Act content is not contributing to the SOTA-parity claim. The contribution of Pre-Act, on Android, is the harness wrapping T3A—not any prompt modification.

5.4.2 Code-Level Runtime Guardrails: Aggregate-Neutral at n=5

We wired an environment variable `PREACT_GUARDRAILS=on/off` that gates three runtime guardrails: (a) double-tap-before-input_text on populated text fields (replacement semantics), (b) image-task no-op cap (force *infeasible* after 4 consecutive no-op actions on image-keyword tasks), and (c) scroll-direction exhaustion notes appended to step summaries. Table 6 reports the n=5 ablation.

Per-task analysis (omitted for space; see findings summary in supplementary materials) reveals task-specific effects that cancel out at the aggregate level. The double-tap guardrail materially helps `AudioRecorderRecordAudioWithFileName`—the task involves a populated filename dialog where vanilla `input_text` appends rather than replaces—but is counterbalanced by minor timing penalties on Camera tasks. The Pre-Act SR claim does *not* depend on these runtime guardrails. We retain them in the production code as edge-case protections, but we do not claim them as contributions in this paper.

5.4.3 State-Machine vs. Flat-Script Runtime

To partially test the architectural commitment that "the SM is the runtime" (§2), we wired an environment variable `PREACT_RUNTIME_MODE=state_machine|flat_script` that gates per-state verification in the replayer. State-machine mode (default) verifies each state's predicate before executing the outgoing transition and falls back to CUA if verification fails. Flat-script mode bypasses verification and executes transitions linearly, mimicking ActionEngine-style flat-Python generation followed by linear execution—both modes share the same compile pipeline and the same stored corpus content; only runtime semantics differ. Table 7 reports the result at n=3 Android seeds, warm runs starting from a 56-program rag_db.

State-machine mode wins by +0.67 tasks on average; all 3 seeds show state-machine $\geq$ flat-script (sign-test $p = 0.125$ at n=3, suggestive but not significant). The wins concentrate on Camera tasks where verification catches mismatches that linear execution misses: e.g., on seed=42 state-machine ran CameraTakePhoto via RPA at cov=100% while flat-script entered an unverified replay that errored, fell through to CUA, and failed in 3 actions. *The architectural commitment matters but is a smaller effect than the verify-gate's 2.6-task diff-of-deltas* (Table 3), consistent with the paper's framing that the gate is the central empirical contribution.

Table 7: Architectural ablation: state-machine vs. flat-script runtime, AndroidWorld official-15 warm, n=3 seeds, identical 56-program rag_db.

Mode	seed=42	seed=100	seed=1337	Mean ± std
state_machine (default)	12/15	12/15	11/15	**11.67 ± 0.58**
flat_script (ActionEngine-style)	11/15	12/15	10/15	**11.00 ± 1.00**
Δ	+1	0	+1	**+0.67 tasks**

Table 8: Validity threats and closure status. Eight of nine in-scope threats closed with multi-seed paper-grade rigor.

#	Threat	Status	Evidence
1	Single-run variance	Closed	n=3 cold→warm + n=5 cold-runs
2	Verify-gate ablation	**Closed cross-platform,** n=5+2	Sign-test $p < 0.001$; OSWorld 50 pp; 5 sm
3	Android cold→warm monotonicity	Closed	n=3 with rag_db reset, all monotonic
4	Prompt-guidance inertness	Closed by codebase state	`agent.py:464` omits `additional_guidel`
5	Code-level guardrails	Closed n=5	Aggregate-neutral (cold means 10.2 = 10.
6	Compile-fidelity taxonomy	Closed	5/5 manual agreement per bucket
7	Step-budget AB	Closed n=5	Within ±2 prediction; wall ratio 0.69
8	Platform-coverage	Out of scope	Requires net-new benchmarks
9	Baseline-parity replication	Closed	Cross-model and cross-seed

5.4.4 Step-Budget Cap: Wall-Time-Only

The dynamic step-budget cap `min(max(20, 8c), 30)` (where c is task complexity) was hypothesized to bound wall time without sacrificing SR. We test against an unbounded 60-step fixed budget at n=5 seeds. The cap-A (current default) mean is 9.8 ± 0.84; cap-B (60-step) mean is 11.6 ± 0.55 (cold runs only). The diff is $+1.8 \pm 0.84$, within the validation-plan ±2 prediction interval. The wall-time ratio (cap-A : cap-B) is 0.69, within the plan's 0.7 prediction. The +1.8 SR is paid for by +30% wall time on the FAIL tail—primarily SystemBrightnessMax, which consumes the full 60-step budget under cap-B and still fails (the failure is harness-deterministic; more budget does not help).

5.5 Summary: 8 of 9 Validity Threats Closed

Table 8 maps each of the nine validity threats from §2 to its closure status.

6 Discussion

6.1 Why the Harness is Load-Bearing

The mechanism is: the verify-gate filters lossy compiles before they enter the corpus. "Lossy" here means: action sequences that are mechanically faithful (cov=100% on replay) but do not produce the live evaluator's required end state. Without the gate, every CUA-success-then-compile cycle adds a new program to the corpus regardless of whether that program actually achieves the goal under deterministic re-execution. With the gate, only programs that have independently re-passed enter the corpus.

This matters because the corpus is the input to subsequent retrieval. A retrieved lossy program will be replayed on a future invocation; the replay will execute mechanically (cov=100%) and the

Table 9: Smoking-gun reproducibility across 5 OSWorld warm-OFF reps. Each row: how many of 5 reps replayed the program at cov=100% and got score=0.

Program ID	Task	Reps with cov=100%/score=0
43188217	LibreOffice Calc formula	5/5 (fully deterministic)
c1f04f87	LibreOffice Calc chart	4/5 (rep2 hybrid-rescued)
8f0fdfa4	Chrome history clean	2/5 (rep3/4/5 replayed correctly)

live evaluator will score 0. Worse, the harness's selector will continue to retrieve the lossy program until something explicitly removes it. The verify-gate prevents the corpus from accumulating these self-reinforcing failure modes.

6.2 The Verify-Gate's Marginal Value is Structural

The most striking quantitative observation in this paper is that the verify-gate's marginal value (Figure 2) is *numerically identical*—2.6 tasks of diff-of-deltas—on two platforms with different LLMs, different task subsets, and different evaluator semantics. This is unlikely under most plausible mechanism alternatives:

- If the gate's value derived from a model-specific behavior (e.g., Gemini-particular flaws), Android's diff-of-deltas would not match OSWorld's (which uses Claude).
- If it derived from task-specific properties (e.g., Markor-edit ambiguity), the desktop-task diff-of-deltas would differ from mobile.
- If it derived from benchmark-evaluator quirks (e.g., Android's specific scoring rubric), OSWorld's evaluators (which score browser state, calc cell formulae, chart properties) would not produce the same magnitude.

Instead, the diff-of-deltas magnitude is structural: it is the rate at which the underlying *compile* step produces lossy state-machine programs. Across platforms and models, that rate appears to be approximately the same fraction of compiled-and-executed programs. The *absolute* regression on OSWorld (43 percentage points on a 6-task subset) is much larger *proportionally* than on Android (17 pp on 15 tasks), but the *absolute number of tasks* affected is the same in both cases. We interpret this as: the lossy-compile rate is a property of the compile step (LLM trace $\rightarrow$ state-machine), not of the platform or evaluator.

6.3 Smoking-Gun Reproducibility

The five smoking-gun cov=100%/score=0 cases (Table 5) are not equally reproducible across reps. Table 9 reports the per-program reproducibility across the 5 OSWorld warm-OFF reps.

The mechanism is partially deterministic. For the most reliable case (Calc formula 43188217), the lossy program reproducibly fails at cov=100% on every warm rep. For Calc chart c1f04f87, hybrid-mode rescue occasionally salvaged the run via CUA fallback after partial replay (cov=8%). For Chrome history 8f0fdfa4, the cold-stored program's mechanical action sequence happened to match the evaluator's required browser state on 3 of 5 reps—a partial-determinism that traces to environmental factors (Chrome's session state, browsing history initialization) varying across DesktopEnv resets even with identical task IDs.

Crucially, the aggregate regression remains fully deterministic. All 5 OFF reps lose at least 2 tasks; no rep escapes the regression. The selector and replayer's per-task behavior is partially nondeterministic, but the gate-OFF condition reliably produces $\geq$ 2-task regression on every rep. This is the property paper claims should rest on, not the per-task smoking guns.

6.4 Harness-Deterministic vs. LLM-Driven Failures

Three AndroidWorld tasks (BrowserDraw, SystemBrightnessMax, SystemWifiTurnOn) and one OSWorld task (LibreOffice Writer macro) fail across all five seeds, both LLM backends (Claude and Gemini), all four guardrail/budget configurations, and both verify-gate conditions. These failures are *harness-deterministic*: they are properties of the UI patterns themselves (status-bar stale reads, scroll-on-seekbar incompatibility, image-canvas-not-in-a11y-tree) rather than properties of the agent's reasoning. They bound the benchmark, not the method.

6.5 Out-of-Distribution Generalization

To test whether the corpus transfers beyond seen tasks, we built a rag_db from the OSWorld test_tiny set (6 tasks across Chrome, LibreOffice Calc, LibreOffice Writer; 4 verify-gate-stored programs after compile filtering) and ran on `test_small_ood.json`: the 33 OSWorld test_small tasks that are *not* in test_tiny, spanning 9 domains (chrome×2, gimp×2, libreoffice_calc×1, libreoffice_impress×2, libreoffice_writer×0, multi_apps×17, os×2, thunderbird×2, vlc×2, vs_code×3). The agentic Program Selector retrieves from the test_tiny corpus when the new task's description plausibly matches.

Result: **warm-OOD 21/30 = 70.0%** (3 tasks errored during environment setup, not counted) vs. a cold-baseline of 26/36 = 72.2% on the full test_small (which includes 6 in-distribution test_tiny tasks; the cold baseline on the 30 OOD tasks alone is approximately 20/30 = 66.7% after subtracting the 6 in-distribution successes).

The cross-platform interpretation: **the test_tiny corpus is approximately neutral on OOD tasks**—warm OOD (≈ 70%) and cold OOD (≈ 67%) differ by 1 task, well within run-to-run noise. Specifically, the corpus does not meaningfully *help* by transferring abstractions across task families (multi_apps, vlc, vs_code, thunderbird, os, gimp), and it does not meaningfully *hurt* by retrieving inapplicable programs and wasting replay budget. The selector tends to return "no candidate" when the OOD task's description does not match a stored program (cov=0% on warm OOD runs is consistent with this). This result clarifies the scope of the "self-extending corpus" claim: **the corpus's value is in repeated invocations of seen tasks, not in cross-task generalization**.

6.6 Limitations

The closure table (Table 8) lists eight closed threats and one out-of-scope. Beyond those, we explicitly acknowledge five remaining limitations:

(L1) Architectural baseline (partial). We ran an ActionEngine-style flat-script baseline at n=3 Android seeds (§5.4.3) by gating per-state verification under a `PREACT_RUNTIME_MODE=flat_script` env var. The state-machine-as-executable mode achieved 11.67 ± 0.58 vs. flat-script's 11.00 ± 1.0 (Android official-15 warm SR), $\Delta = +0.67$ tasks in favor of state-machine, with all 3 seeds showing state-machine $\geq$ flat-script (sign-test $p = 0.125$ at n=3, suggestive but not significant). The architectural commitment matters but is a smaller effect than the verify-gate's 2.6-task diff-of-deltas, consistent with the paper's framing that the gate is the bigger empirical contribution. We did not run an untargeted-exploration baseline.

(L2) Benchmark coverage. OSWorld test_tiny has only 6 tasks, AndroidWorld official-15 has 15. While the verify-gate effect replicates cross-platform, the benchmarks themselves are a small sample of computer-using tasks. We acknowledge platform-coverage bias (#8 in Table 8) as out-of-scope for this paper. The relative magnitude of the OSWorld effect is also influenced by the small denominator: 50 pp on a 6-task subset is mathematically larger than 17 pp on 15 tasks for the same absolute number of regressions.

(L3) Compile cost. The verify-before-store gate requires a full re-replay and re-evaluation of every compiled program. From timing measurements across our experiments (60 OSWorld tasks; 60 Android tasks under gate-ON), the verify-replay phase adds median 141 s on OSWorld and 76 s on Android per stored task, on top of the original CUA execution (median 65 s OSWorld; 47 s Android). This corresponds to **+70% wall overhead on OSWorld and +134% on Android per successful CUA-then-compile cycle** (the relative figure is higher on Android because Android CUA is faster, so the fixed-cost verify-replay dominates more proportionally). The cost is amortized on warm runs where the corpus retrieves already-verified programs in seconds, but the compile-time overhead is real and may matter for high-throughput deployments. The double-gate cost is the price of monotonicity: §5.3's diff-of-deltas of 2.6 tasks on each platform is paid for in this verify-replay budget.

(L4) Compile-fidelity rate. Although our taxonomy audit (§5) found that compile-fidelity issues affect roughly 28% of Android programs (16/58) and similar fractions on OSWorld, the compiler itself is an LLM that produces lossy state-machines on a non-trivial fraction of inputs. The verify-gate filters this at storage time, but as the corpus scales, the gate's verify-replay cost (which is proportional to the number of compile attempts, not just stored programs) may become a bottleneck.

(L5) LLM nondeterminism in selector. The agentic Program Selector is itself an LLM call. We measured selector exact-id-match accuracy at $22/30 = 73\%$ on the rag_db's stored programs, prompted with each program's own task description verbatim. Inspecting the 8 misses, all were picks of *semantically equivalent* programs (same `task_description`, different `program_id` from accumulated reps under different dedup signatures). Functional accuracy (correct task family selected) is therefore close to 100%, with the 73% exact-id figure reflecting the corpus's accumulation of multiple matches per task. Selector nondeterminism on novel-task discrimination remains future work.

7 Conclusion

PreAct demonstrates that a *verified self-extending executable code corpus* produces monotonic refinement on multi-platform CUA benchmarks. The harness—RAG retrieval, verify-before-store gate, agentic program selector, hybrid replay, and CUA fallback—is the load-bearing contribution: cross-platform n=5+2 ablation shows the gate is empirically necessary for monotonicity, with five reproducible smoking-gun replays documenting the lossy-compile pattern it filters. Equally informative are the *negative* findings: code-level runtime guardrails are aggregate-neutral, and prompt-level Pre-Act content is dead code in production. The harness is what works; the runtime add-ons are not load-bearing.

The architectural commitment that makes this possible is *the artifact is the runtime*: state-machine programs in PreAct's corpus are not flat scripts generated from a state machine but the directly-executable graphs themselves. What the agent "remembers" is what it can directly execute, and the corpus self-extends through verified compile cycles.

Future work falls in three directions:

Scaling the corpus. The current corpus (58 Android programs after multi-week experiments) is small. We have not characterized the corpus's behavior at 10^3 or 10^4 programs: does the agentic selector still discriminate accurately when the candidate set is large? Does dedup-signature collision become an issue? Does the verify-replay step cost become prohibitive at scale?

Architectural baselines. The state-machine-as-executable claim deserves a direct AB against a flat-script baseline (e.g., an ActionEngine-style implementation that runs the same compile cycle but generates flat Python at the verify step). The current paper supports the

claim by working implementation across two platforms but not by direct comparison.

Long-horizon tasks. Both AndroidWorld and OSWorld have task horizons of order 10–30 actions. Tasks requiring long-term planning, goal decomposition, or recovery from compound failures (e.g., entire workflow tasks lasting hundreds of actions) are not represented in our benchmarks. We expect the harness's CUA-fallback mechanism to be tested most severely on such tasks; integration with explicit goal-decomposition planners is the natural next step.

References

[1] Anonymous. AgentRR: Multi-level experience replay for agents. *arXiv preprint arXiv:2505.17716*, 2025.

[2] Anonymous. Compiled AI: Compiling agent workflows into deterministic code. Working manuscript, 2025.

[3] Anonymous. ActionEngine: State machine discovery via app crawling. *arXiv preprint arXiv:2602.20502*, 2026. Microsoft Research.

[4] Anthropic. Introducing computer use. https://www.anthropic.com/news/3-5-models-and-computer-use, 2024.

[5] Browser-Use. Workflow-use: Linear script compilation with agent fallback. https://github.com/browser-use/workflow-use, 2025.

[6] Saideep Chundru. The rerun crisis in computer using agents, 2026. Working manuscript.

[7] Pig.dev. Muscle-mem: Tool-call replay with agent fallback. https://github.com/pig-dot-dev/muscle-mem, 2025.

[8] Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, et al. AndroidWorld: A dynamic benchmarking environment for autonomous agents. *ICLR*, 2025.

[9] Yongchao Wang et al. SAGE: Skill-augmented agent architecture. Working manuscript, 2024.

[10] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *NeurIPS Datasets and Benchmarks Track*, 2024.

[11] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *ICLR*, 2023.

Appendix A. Computer-Action Schema

Each transition's `action` field must be one of the action types in Table 10. Action types are translated to platform-specific backends at execution time: `action_click` on Android maps to a `JSONAction`-formatted touch with the resolved element's (x, y) coordinates; on OSWorld it maps to `pyautogui.click(x, y)`. The cross-platform schema enables the same compiled state-machine program to run on either backend, although in practice each task's compile is platform-specific because the verification predicates use platform-specific selector dialects (Android XPath vs. pyautogui screenshot regions).

Table 10: Computer action types in PreAct's program schema. Each transition uses exactly one type.

Action type	Description and required fields
`action_click`	Click on a UI element. `target`: selector.
`action_long_press`	Long-press on a UI element. `target`: selector.
`input_text`	Type text into a focused element. `text`: literal or $param. `index`: optional element id.
`action_keypress`	Send a key event. `key`: `Enter`, `Back`, `Tab`, etc.
`scroll`	Scroll a region. `direction`: `up/down/left/right`. Optional `index` for sub-element scroll.
`open_app`	Launch an application by name. `app_name`: string.
`wait`	Sleep for the platform's default delay.
`navigate_back`	Send the system Back gesture (Android) or browser-back equivalent.
`navigate_home`	Return to the home screen / desktop.
`inspect_text`	Read the value at a selector. `store_result_as`: variable name. Used by QA-style tasks.
`answer`	Emit a final textual answer. `text`: literal or computed.
`status`	Terminate the program. `goal_status`: `complete/infeasible`.

The `description` field on each transition (one short sentence) is consumed by the agentic Program Selector (§3) when judging whether a stored program applies to the current task. Selectors are written to be parameter-aware (e.g. "type the filename (parameter `filename`)" rather than "type text"), since the program's `parameters` list is what the LLM uses to bind the program's slots to the new task's instance values at retrieval time.

Appendix B. Per-Task Multi-Seed Data

Table 11 reports per-task warm SR across the three Gemini 3 Flash seeds used for the cold→warm monotonicity experiment (§5.2). The stable failure set (BrowserDraw, SystemBrightnessMax, SystemWifiTurnOn) is observed across all three seeds. The variance task (BrowserMaze) is the maze-pathfinding task, where the agent must navigate a grid by directional clicks and the action sequence is sensitive to LLM sampling.

The complete per-task results across all 32 Android multi-seed runs and 8 OSWorld runs are also available in the project's `paper/findings_summary.md`.

Appendix C. Container Patch

The `android_server_patched.py` adds a `/state` endpoint to `huggingface/android_world:latest` that returns base64-encoded screenshots plus serialized accessibility-tree elements, sidestepping the upstream populated-AVD a11y-gRPC initialization wedge.

Table 11: Per-task pass/fail across three Gemini 3 Flash seeds on AndroidWorld official-15 (warm-pass results).

Task	seed=42	seed=100	seed=1337
AudioRecorderRecordAudio	PASS	PASS	PASS
AudioRecorderRecordAudioWithFileName	PASS	PASS	PASS
BrowserDraw	FAIL	FAIL	FAIL
BrowserMaze	FAIL	PASS	FAIL
CameraTakePhoto	PASS	PASS	PASS
CameraTakeVideo	PASS	PASS	PASS
ClockStopWatchPausedVerify	PASS	PASS	PASS
ClockStopWatchRunning	PASS	PASS	PASS
ContactsAddContact	PASS†	PASS†	PASS†
ContactsNewContactDraft	PASS	PASS	PASS
FilesDeleteFile	PASS	PASS	FAIL
MarkorCreateFolder	PASS†	PASS†	PASS†
MarkorCreateNote	PASS	PASS	PASS
SystemBrightnessMax	FAIL	FAIL	FAIL
SystemWifiTurnOn	FAIL	FAIL	FAIL
Total	11/15	12/15	10/15

† Live evaluator passes but the verify-before-store gate rejected the compiled program (`verify_score=0`).